

Skill Week 5

1. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation



```
GNU nano 8.7.1 src/directory_navigation.c *
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <limits.h>

int main() {
    char path[PATH_MAX];
    char previous[PATH_MAX] = "";

    while (1) {
        printf("\nCurrent directory: ");
        getcwd(path, sizeof(path));
        printf("%s\n", path);

        printf("Enter directory path (or 'prev' or 'exit'): ");

        char input[PATH_MAX];
        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0) {
            break;
        }

        if (strcmp(input, "prev") == 0) {
            if (strlen(previous) == 0) {
                printf("No previous directory.\n");
            } else {
                char temp[PATH_MAX];
                strcpy(temp, path);

                if (chdir(previous) == 0) {
                    strcpy(previous, temp);
                    printf("Moved to previous directory.\n");
                } else {
                    perror("chdir");
                }
            }
        } else {
            char temp[PATH_MAX];
            strcpy(temp, path);

            if (chdir(input) == 0) {
                strcpy(previous, temp);
                printf("Directory changed successfully.\n");
            } else {
                perror("chdir");
            }
        }
    }

    return 0;
}
```

OUTPUT:

```
root@DUDDUKURIs x + v
mkdir -p Skill-Week-5/src Skill-Week-5/bin
cd Skill-Week-5
root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ pwd
/home/DUDDUKURIs/OS_Skill/Skill-Week-5
root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ nano src/directory_
navigation.c

root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ gcc src/directory_n
avigation.c -o bin/directory_navigation
root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ ./bin/directory_nav
igation

Current directory: /home/DUDDUKURIs/OS_Skill/Skill-Week-5
Enter directory path (or 'prev' or 'exit'): .
Directory changed successfully.

Current directory: /home/DUDDUKURIs/OS_Skill/Skill-Week-5
Enter directory path (or 'prev' or 'exit'): ..
Directory changed successfully.

Current directory: /home/DUDDUKURIs/OS_Skill/Skill-Week-5
Enter directory path (or 'prev' or 'exit'): /tmp
Directory changed successfully.
```

```
Current directory: /tmp
Enter directory path (or 'prev' or 'exit'): prev
Moved to previous directory.

Current directory: /home/DUDDUKURIs/OS_Skill
Enter directory path (or 'prev' or 'exit'): exit
```

2. To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
GNU nano 8.7.1 src/builtin_dispatch.c *
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>

void pwd_command() {
    char path[1024];

    if (getcwd(path, sizeof(path)) != NULL)
        printf("%s\n", path);
    else
        perror("pwd");
}

void cd_command(char *path) {
    if (path == NULL) {
        printf("cd: missing argument\n");
        return;
    }

    if (chdir(path) != 0)
        perror("cd");
}
```

```

void exit_command() {
    printf("Exiting shell...\n");
    exit(0);
}

struct builtin {
    char *name;
    void (*function)();
};

struct builtin commands[] = {
    {"pwd", pwd_command},
    {"exit", exit_command}
};

```

OUTPUT:

```

root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ nano src/builtin_dispatch.c

root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ gcc src/builtin_dispatch.c -o bin/builtin_dispatch
root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ ./bin/builtin_dispatch
myshell> pwd
/home/DUDDUKURIs/OS_Skill/Skill-Week-5
myshell> cd/tmp
Unknown command: cd/tmp
myshell> cd /tmp
myshell> hello
Unknown command: hello
myshell> exit
Exiting shell...

```

3.To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```
root@DUDDUKURIs  ×  +  v
GNU nano 8.7.1 src/shell_exit.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main() {
    char path[1024];
    char input[100];

    if (getcwd(path, sizeof(path)) != NULL)
        printf("Current directory: %s\n", path);
    else
        perror("getcwd");

    FILE *file = fopen("shell_state.txt", "w");

    if (file == NULL) {
        perror("fopen");
        return 1;
    }

    fprintf(file, "Last directory: %s\n", path);

    while (1) {
        printf("myshell> ");

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0) {
            fprintf(file, "Shell exited successfully.\n");
            fclose(file);

            printf("State saved. Resources cleaned up.\n");
            printf("Exiting shell...\n");

            exit(0);
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

OUTPUT:

```
root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ cd ~/OS_Skill/Skill-Week-5
root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ nano src/shell_exit.c

root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ gcc src/shell_exit.c -o bin/shell_exit
root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ ./bin/shell_exit
Current directory: /home/root/OS_Skill/Skill-Week-5
myshell> hello
You entered: hello
myshell> exit
State saved. Resources cleaned up.
Exiting shell...
root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ cat shell_state.txt
Last directory: /home/root/OS_Skill/Skill-Week-5
Shell exited successfully.
```

4. To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```
GNU nano 8.7.1 src/export_command.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

int valid_name(char *name) {
    if (!isalpha(name[0]) && name[0] != '_')
        return 0;

    for (int i = 1; name[i] != '\0'; i++) {
        if (!isalnum(name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

int main() {
    char input[200];

    while (1) {
        printf("myshell> ");

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strcmp(input, "export ", 7) == 0) {
            char *assignment = input + 7;
            char *equal = strchr(assignment, '=');

            if (equal == NULL) {
                printf("Invalid export syntax.\n");
                continue;
            }

            *equal = '\0';

            char *name = assignment;
            char *value = equal + 1;

            if (!valid_name(name)) {
                printf("Invalid variable name.\n");
                continue;
            }

            setenv(name, value, 1);

            printf("Exported %s=%s\n", name, value);
        }
        else if (strcmp(input, "print ", 6) == 0) {
            char *value = getenv(input + 6);

            if (value != NULL)
                printf("%s=%s\n", input + 6, value);
            else
                printf("Variable not found.\n");
        }
        else {
            printf("Unknown command.\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ nano src/export_command.c
root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ gcc src/export_command.c -o bin/export_command
root@DUDDUKURIs:~/OS_Skill/Skill-Week-5$ ./bin/export_command
myshell> export NAME=Charan
Exported NAME=Charan
myshell> export NAME=Charan
Exported NAME=Charan
myshell> print NAME
NAME=Charan
myshell> export NAME=Student
Exported NAME=Student
myshell> print NAME
NAME=Student
```